

DBMS ASSIGNMENT

Neha Voma 24mab0a44

1.

WITH new_table AS (

SELECT

d.name AS Department,

e.name AS Employee,

e.salary AS Salary,

DENSE_RANK() OVER (

PARTITION BY d.name

ORDER BY e.salary DESC

) AS Ranking

FROM Employee e

LEFT JOIN Department d

ON e.departmentId = d.id

)

SELECT

Department,

Employee,

Salary

FROM new_table

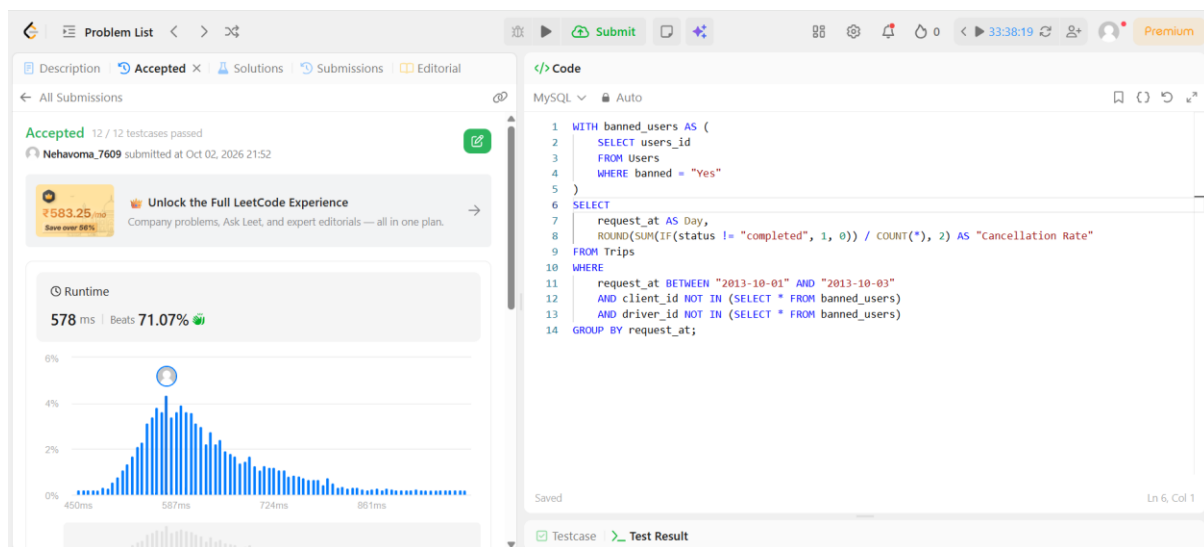
WHERE Ranking <= 3;

The screenshot shows a LeetCode problem solution for 'Department Highest Salary'. The interface includes a problem list, a description tab, and a code editor. The code is a MySQL query that uses a CTE named 'new_table' to calculate the dense rank of employees by department. The query selects the department name, employee name, and salary, ordered by department and then by salary in descending order. The results are limited to the top 3 employees in each department. The runtime is 961 ms, which beats 83.43% of other solutions. A histogram shows the distribution of runtime times, with the user's solution being in the top 15%.

```
1 WITH new_table AS (  
2     SELECT  
3         d.name AS Department,  
4         e.name AS Employee,  
5         e.salary AS Salary,  
6         DENSE_RANK() OVER (  
7             PARTITION BY d.name  
8             ORDER BY e.salary DESC  
9         ) AS Ranking  
10    FROM Employee e  
11    LEFT JOIN Department d  
12        ON e.departmentId = d.id  
13 )  
14  
15 SELECT  
16     Department,  
17     Employee,  
18     Salary  
19 FROM new_table  
20 WHERE Ranking <= 3;
```

2.

```
WITH banned_users AS (  
    SELECT users_id  
    FROM Users  
    WHERE banned = "Yes"  
)  
SELECT  
    request_at AS Day,  
    ROUND(SUM(IF(status != "completed", 1, 0)) / COUNT(*), 2) AS "Cancellation Rate"  
FROM Trips  
WHERE  
    request_at BETWEEN "2013-10-01" AND "2013-10-03"  
    AND client_id NOT IN (SELECT * FROM banned_users)  
    AND driver_id NOT IN (SELECT * FROM banned_users)  
GROUP BY request_at;
```



3.

```
SELECT id,visit_date,people  
FROM stadium  
WHERE id IN (  
    SELECT s1.id  
    FROM stadium s1  
    JOIN stadium s2 ON s1.id=s2.id-1  
    JOIN stadium s3 ON s2.id=s3.id-1
```

WHERE s1.people>=100 AND s2.people>=100 AND s3.people>=100

UNION

SELECT s2.id

FROM stadium s1

JOIN stadium s2 ON s1.id=s2.id-1

JOIN stadium s3 ON s2.id=s3.id-1

WHERE s1.people>=100 AND s2.people>=100 AND s3.people>=100

UNION

SELECT s3.id

FROM stadium s1

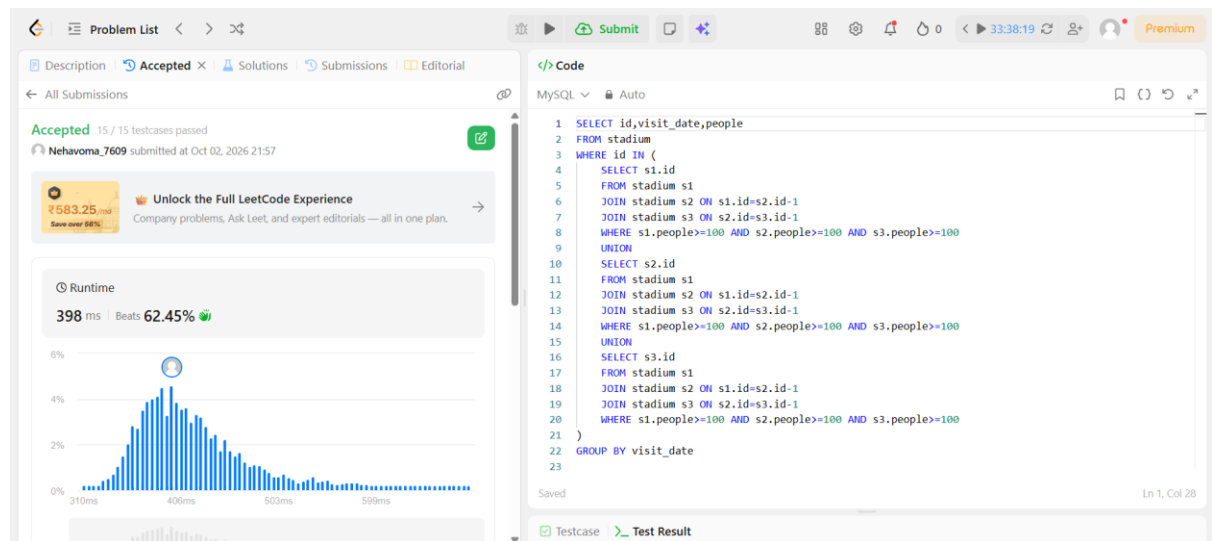
JOIN stadium s2 ON s1.id=s2.id-1

JOIN stadium s3 ON s2.id=s3.id-1

WHERE s1.people>=100 AND s2.people>=100 AND s3.people>=100

)

GROUP BY visit_date



4.

WITH RECURSIVE

cte AS (

SELECT

employee_id,

employee_name,

1 AS level

FROM Employees

WHERE manager_id IS NULL

```

        UNION ALL

SELECT
    e.employee_id,
    e.employee_name,
    c.level + 1 AS level
FROM Employees e
JOIN cte c
    ON e.manager_id = c.employee_id
),
cte1 AS (
    SELECT
        employee_id AS manager_id,
        employee_id,
        salary
    FROM Employees
    UNION ALL
    SELECT
        c1.manager_id,
        e.employee_id,
        e.salary
    FROM cte1 c1
    JOIN Employees e
        ON c1.employee_id = e.manager_id
),
cte2 AS (
    SELECT
        c.manager_id AS employee_id,
        COUNT(*) - 1 AS team_size,
        SUM(c.salary) AS budget
    FROM cte1 c
    GROUP BY 1
)
SELECT
    c.employee_id,
    c.employee_name,
    c.level,
    c2.team_size,

```

c2.budget

FROM cte2 c2

LEFT JOIN cte c

ON c.employee_id = c2.employee_id

ORDER BY 3, 5 DESC, 2;

The screenshot displays the LeetCode submission page for a MySQL problem. On the left, the 'All Submissions' tab is active, showing a submission by 'Nehavoma_7609' that was 'Accepted' on Oct 02, 2026, at 22:27. Below this, a bar chart shows the distribution of execution times, with a peak around 39ms. On the right, the 'Code' editor shows a MySQL query using recursive CTEs to split a string into words and then process them. The query is as follows:

```
1 WITH RECURSIVE split AS (  
2     SELECT  
3         content_id,  
4         1 AS pos,  
5         SUBSTRING_INDEX(content_text, ' ', 1) AS word,  
6         IF(LOCATE(' ', content_text) = 0, NULL,  
7           SUBSTRING(content_text, LOCATE(' ', content_text) + 1)) AS rest  
8     FROM user_content  
9     UNION ALL  
10    SELECT  
11        content_id,  
12        pos + 1,  
13        SUBSTRING_INDEX(rest, ' ', 1),  
14        IF(LOCATE(' ', rest) = 0, NULL,  
15          SUBSTRING(rest, LOCATE(' ', rest) + 1))  
16    FROM split  
17    WHERE rest IS NOT NULL  
18 ),  
19 conv AS (  
20     SELECT  
21         content_id,  
22         pos,  
23         word  
24 )
```

5.

WITH ipseparated AS (

SELECT

ip,

CASE

WHEN CAST(SUBSTRING_INDEX(ip, ' ', 1) AS UNSIGNED) > 255

THEN 'invalid'

ELSE 'valid'

END AS part1,

CASE

WHEN LENGTH(SUBSTRING_INDEX(SUBSTRING_INDEX(ip, ' ', 2), ' ', -1)) > 1

AND SUBSTRING_INDEX(SUBSTRING_INDEX(ip, ' ', 2), ' ', -1)

REGEXP '^0[0-9]+'

or CAST(SUBSTRING_INDEX(SUBSTRING_INDEX(ip, ' ', 2), ' ', -1) AS UNSIGNED) > 255

THEN 'invalid'

ELSE 'valid'

END AS part2,

CASE

WHEN LENGTH(SUBSTRING_INDEX(SUBSTRING_INDEX(ip, ' ', 3), ' ', -1)) > 1

AND SUBSTRING_INDEX(SUBSTRING_INDEX(ip, ' ', 3), ' ', -1)

```

        REGEXP '^0[0-9]+'
```

```

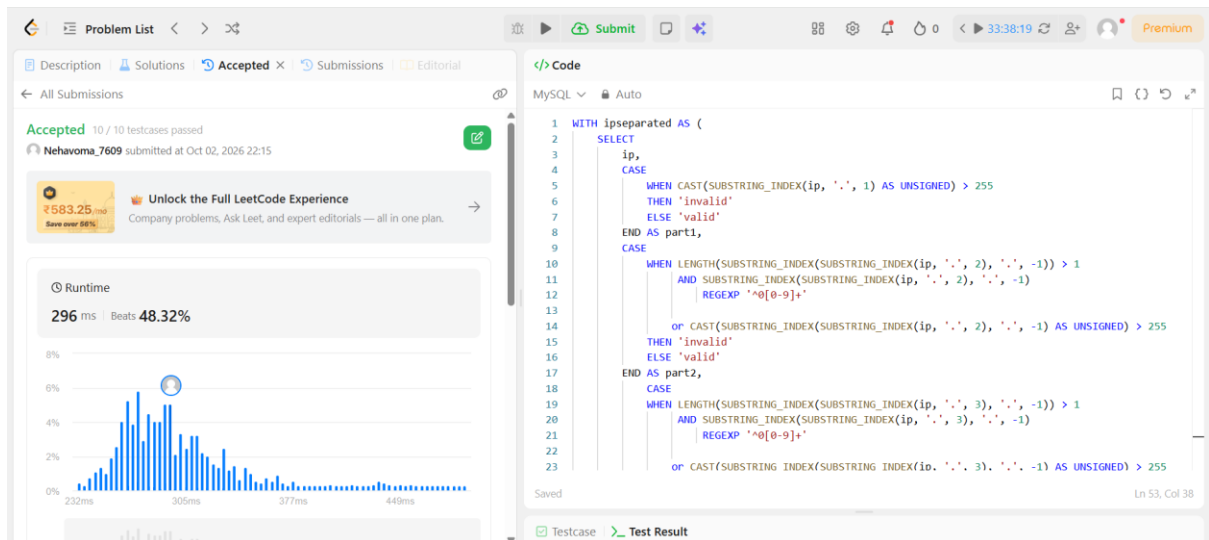
        or CAST(SUBSTRING_INDEX(SUBSTRING_INDEX(ip, '.', 3), '.', -1) AS UNSIGNED) > 255
    THEN 'invalid'
    ELSE 'valid'
END AS part3,

CASE
    WHEN LENGTH(SUBSTRING_INDEX(ip, '.', -1)) > 1
        AND SUBSTRING_INDEX(ip, '.', -1)
            REGEXP '^0[0-9]+'
        or CAST(SUBSTRING_INDEX(ip, '.', -1) AS UNSIGNED) > 255
    THEN 'invalid'
    ELSE 'valid'
END AS part4,

COUNT(*) AS invalid_count
FROM logs
GROUP BY ip
)

select ip , invalid_count
from ipseparated
where
(LENGTH(ip) - LENGTH(REPLACE(ip, '.', ''))) <> 3
or
part1 = "invalid"
or
part2 = "invalid"
or part3 = "invalid"
or part4 = "invalid"
order by invalid_count desc , ip desc

```



6.

WITH RECURSIVE

cte AS (

SELECT

employee_id,

employee_name,

1 AS level

FROM Employees

WHERE manager_id IS NULL

UNION ALL

SELECT

e.employee_id,

e.employee_name,

c.level + 1 AS level

FROM Employees e

JOIN cte c

ON e.manager_id = c.employee_id

),

cte1 AS (

SELECT

employee_id AS manager_id,

employee_id,

salary

FROM Employees

UNION ALL

SELECT

c1.manager_id,

e.employee_id,

e.salary

FROM cte1 c1

JOIN Employees e

ON c1.employee_id = e.manager_id

),

cte2 AS (

SELECT

c.manager_id AS employee_id,

COUNT(*) - 1 AS team_size,

SUM(c.salary) AS budget

FROM cte1 c

GROUP BY 1

)

SELECT

c.employee_id,

c.employee_name,

c.level,

c2.team_size,

c2.budget

FROM cte2 c2

LEFT JOIN cte c

ON c.employee_id = c2.employee_id

ORDER BY 3, 5 DESC, 2;

Accepted 11 / 11 testcases passed
Nehavoma_7609 submitted at Oct 02, 2026 22:18

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime
304 ms Beats 81.29%

MySQL

```

1 WITH RECURSIVE
2 cte AS (
3     SELECT
4         employee_id,
5         employee_name,
6         1 AS level
7     FROM Employees
8     WHERE manager_id IS NULL
9
10    UNION ALL
11
12    SELECT
13        e.employee_id,
14        e.employee_name,
15        c.level + 1 AS level
16    FROM Employees e
17    JOIN cte c
18        ON e.manager_id = c.employee_id
19 ),
20 cte1 AS (
21     SELECT
22         employee_id AS manager_id,
23         employee_id

```

7.

WITH

filter AS(

SELECT

user_id

,category

FROM productpurchases p1

INNER JOIN productinfo p2

ON p1.product_id=p2.product_id

)

SELECT

f1.category AS category1

,f2.category AS category2

,COUNT(DISTINCT f1.user_id) AS customer_count

FROM filter f1

INNER JOIN filter f2

ON f1.user_id=f2.user_id

AND f1.category<f2.category

GROUP BY f1.category, f2.category

HAVING COUNT(DISTINCT f1.user_id)>=3

ORDER BY customer_count DESC, f1.category, f2.category

Problem List

Accepted

Submissions

Editorial

← All Submissions

Accepted 12 / 12 testcases passed

Nehavoma_7609 submitted at Oct 02, 2026 22:20

₹583.25

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

632 ms Beats 85.92%



Code

MySQL Auto

```
1 WITH
2 filter AS(
3     SELECT
4         user_id
5         ,category
6     FROM productpurchases p1
7     INNER JOIN productinfo p2
8         ON p1.product_id=p2.product_id
9 )
10 SELECT
11     f1.category AS category1
12     ,f2.category AS category2
13     ,COUNT(DISTINCT f1.user_id) AS customer_count
14 FROM filter f1
15 INNER JOIN filter f2
16     ON f1.user_id=f2.user_id
17     AND f1.category<f2.category
18 GROUP BY f1.category, f2.category
19 HAVING COUNT(DISTINCT f1.user_id)>=3
20 ORDER BY customer_count DESC, f1.category, f2.category
21
```

Saved Ln 21, Col 1

Testcase Test Result